

Testing the model

Continuing with this program, we will use our test data from *optdigits-test.csv* to see how the predictions from our model turn out. Here's how we can test the predictions. Add the following line in the *Main()* method:

```
Test(env);
```

Then, add the following *Test()* method to your project:

```
public static void Test(LocalEnvironment env)
{
    ITransformer loadedModel;
    using (var f = new FileStream(_modelpath,
        FileMode.Open))
        loadedModel = TransformerChain.LoadFrom(env, f);

    using (var reader = new StreamReader(_testdatapath))
    {
        int iCount = 0, cCount = 0;

        while (!reader.EndOfStream)
        {
            var line = reader.ReadLine();
            var pixels = line.Substring(0, line.Length - 2);
            var number = line.Substring(line.Length - 1, 1);

            var testValue = Array.ConvertAll(pixels.
                Split(','), float.Parse);
            var predictor = loadedModel.MakePredictio
                nFunction<MNISTData, NumberPrediction>(env);
            var prediction = predictor.Predict(new
                MNISTData() { PixelValues = testValue, Number = number });
            var predictedNum = int.Parse(prediction.NumberLabel);

            if (predictedNum != uint.Parse(number))
            {
                iCount++;
                Console.WriteLine($"{iCount}
Incorrect prediction {predictedNum} for {number}");
            }
            else
            {
                cCount++;
                Console.WriteLine($"{cCount} Correct
prediction {predictedNum} for {number}");
            }
        }
        Console.WriteLine($"Incorrect predictions:
{iCount.ToString()}");
        Console.WriteLine($"Correct predictions:
{cCount.ToString()}");
    }
}
```

In this method, we are just reading our test data set and converting the values found in each line according to the *MNISTData* input data class defined earlier. The prediction is returned as a predicted label and since the return value is a string, we just convert this value to *int* type to get the actual number this model is predicting.

In the rest of code in this method, we are just comparing the predicted value against the actual number in the last column of our data set to see if our predictions are correct, and writing them to the console to make it easier to see how our model is performing. The following code is a truncated output from one of our executions, for your reference:

```
$ cd mnist
$ dotnet run
...
...
1713 Correct prediction 8 for 8
1714 Correct prediction 9 for 9
1715 Correct prediction 8 for 8
Incorrect predictions: 82
Correct predictions: 1715
```

Based on the total number of samples (1797) in our test data set, the above numbers indicate an accuracy of 95.43 per cent, which is a good number to have.

We have taken care to ensure you are able to follow the steps in this article, build this model from scratch and test it successfully.

Going forward, you could find ways to push the limits and increase the accuracy of this model. Some of the ways you can do that is by trying out a different classifier and/or using the higher resolution MNIST data set. **END** 🐧

*This dataset was made available courtesy of E. Alpaydin and C. Kaynak, Department of Computer Engineering, Bogazici University, 80815, Istanbul, Turkey. We would also like to acknowledge Dua, D. and Karra Taniskidou, E. (2017). UCI Machine Learning Repository [http://archive.ics.uci.edu/ml]. Irvine, CA: University of California, School of Information and Computer Science.

By: Sagar Bhanudas Joshi and Ashish Sahu

Sagar Bhanudas Joshi has been working with developers to help embrace the Universal Windows Platform and the Microsoft Azure platform for more than six years. His current role includes helping SaaS companies architect, design and on-board solutions to Microsoft Cloud Platform, with special focus on machine learning and AI. Joshi lives and works in Mumbai, India. You can contact him on Twitter: [@sagarjms](#)

Ashish Sahu is a senior technology evangelist, working with Microsoft India and helping ISVs and startups overcome technical challenges, adopt the latest technologies, and evolve their solutions to the next level. His Twitter handle is [@ashisa_](#)